

Compatibility Changes in MongoDB 9.0

Important: MongoDB 9.0 Availability

MongoDB 9.0 is production ready and is being incrementally rolled out to MongoDB Atlas clusters that use the Latest Version With Auto Upgrades option.

Availability for the remainder of MongoDB Atlas, MongoDB Enterprise Advanced, and MongoDB Community is coming soon.

This page describes changes introduced in MongoDB 9.0 that can affect compatibility with older versions of MongoDB.

Deprecations

Deprecated	Description
<code>prefixPreview</code> , <code>suffixPreview</code> , <code>substringPreview</code>	Starting in MongoDB 9.0, the <code>prefixPreview</code> , <code>suffixPreview</code> , and <code>substringPreview</code> query types using the <code>text</code> algorithm are deprecated and removed. Use <code>prefix</code> , <code>suffix</code> , or <code>substring</code> query types with the <code>string</code> algorithm instead.
<code>fileDisableSubstringPreviewParameterLimits</code>	You cannot override the internal limits on substring queries against encrypted fields in MongoDB 9.0.

Aggregation

Empty Field Names in `$group`

Starting in MongoDB 9.0, the `$group` stage returns an error if an accumulator expression has an empty field name. For details, see [Empty Field Name \\$group Restriction](#).

`allowPartialResults` in the aggregate `$queryStats` Key

Starting in MongoDB 9.0, the `$queryStats` key for `aggregate` commands includes the `allowPartialResults` option when the option is explicitly set. Query stats can then distinguish between requests where `allowPartialResults` is omitted, explicitly `true`, or explicitly `false`.

The addition changes the `key` and `keyHash` serialization for `aggregate` queries that set `allowPartialResults`. The `queryShapeHash` is unchanged. Downstream consumers that match on `keyHash` might observe different values for these queries.

For more information, see the [aggregate command query shape](#).

Query Language

Null Comparisons on Dotted Paths That Traverse Arrays

Starting in MongoDB 9.0, a dotted path that does not resolve to a non-null value evaluates as `null`. The new behavior applies when a field in the path holds an empty array, an array of scalar values, or an array that contains a nested array. In earlier versions, these paths did not evaluate as `null`, which produced results that disagreed with `$exists`.

Consider a collection that contains the following documents:

```
{ _id: 1, a: [ 1 ] }
{ _id: 2, a: [ ] }
```

The path `a.b` does not resolve to a non-null value in either document. Starting in MongoDB 9.0, the `{ "a.b": null }` query matches both documents. In earlier versions, the query matched neither document. The `{ "a.b": { $ne: null } }` query returns the opposite results: in MongoDB 9.0, the query matches neither document, and in earlier versions, it matched both.

MongoDB does not traverse into nested arrays, so an element that is itself an array never resolves the rest of the path. Consider a collection that contains the following documents:

```
{ _id: 3, a: [ [ { b: 3 } ] ] }
{ _id: 4, a: [ [ { b: 2 } ], { b: 3 } ] }
```

Each `b` value in `_id: 3` and the `b` value of `2` in `_id: 4` sit inside a nested array, so the path `a.b` does not reach them. Starting in MongoDB 9.0, the `{ "a.b": null }` query matches both documents, and the `{ "a.b": { $ne: null } }` query matches neither. In earlier versions, the `{ "a.b": null }` query matched neither document. The nested array causes `_id: 4` to match `{ "a.b": null }` even though its second element resolves `a.b` to the non-null value `3`.

The new behavior affects comparisons to `null` that use the `$eq`, `$ne`, `$in`, `$nin`, `$gte`, and `$lte` operators. Equality matching in the `$lookup` stage follows the same semantics.

Upgrade Impact

Before you upgrade to MongoDB 9.0, review queries and `$lookup` stages that compare a dotted path to `null`. If a field in the path holds an empty array, an array of scalar values, or an array that contains a nested array, these queries return different result sets after the upgrade. Queries that use `{ $ne: null }` to find documents where a dotted path exists and is not `null` return fewer documents, and queries that use `{ $eq: null }` return more documents.

General Changes

Queries Fail When an Indexed Field Becomes Multikey

Starting in MongoDB 9.0, if an indexed field becomes a `multikey` field while a query that references the field is running, the query might fail with a `QueryKilledError`. An indexed field becomes multikey when you insert or update a document so the field contains an array value.

If your query fails with this error, re-run the query after the insert or update operation completes.

Change Streams Enforce Read Preference After Elections

Starting in MongoDB 9.0, a change stream opened with a `readPreference` of `primary` or `secondary` returns the resumable error `InterruptedDueToReplStateChange` (error code 11602) from `getMore` if a replica set election changes the node's role so that it no longer satisfies the read preference. In earlier versions, the cursor kept returning results from the same node.

Compliant drivers and `mongos` resume automatically from the last resume token. Manual `getMore` loops and `mongosh` must resume with `resumeAfter`. For details, see [Resume a Change Stream](#).

Geospatial Index Key-Extraction Error Codes

Starting in MongoDB 9.0, `2dsphere` index key-extraction failures return the named error codes `510` (`GeoKeyExtractionFailed`) and `511` (`GeoKeyExtractionFailedTimeseries`). These codes replace the previous assertion codes `16755` and `16756` for regular `2dsphere` indexes and `183934` and `183493` for time series collections. If your application matches on the previous codes, update it to match `510` and `511`.

Stronger Collection Locks for Cross-Database `renameCollection`

Starting in MongoDB 9.0, when you rename a collection between different databases on a replica set, the `renameCollection` command holds an exclusive lock on the source and target collections. The lock lasts for the entire operation, and blocks `DDL operations` and writes to both collections. Most read operations use `lock-free reads` and are not blocked.

Earlier versions released the source collection's lock before `renameCollection` completed the rename. A concurrent write to the source collection during that window could be lost.

The change affects replica sets only. Sharded clusters already lock both collections for the duration of the rename.

Server-Side JavaScript Date Operations Use UTC

Starting in MongoDB 9.0, server-side JavaScript runs on a WebAssembly (WASM)-based engine that evaluates local `Date` operations in UTC, regardless of the `mongod` host time zone. In earlier versions, these operations observed the host time zone. The change affects JavaScript that runs in `$function`, `$accumulator`, `$where`, and `mapReduce`.

The stored BSON date value is unchanged, as are UTC operations such as `Date.prototype.getTime()`, `Date.prototype.toISOString()`, and the `getUTC*()` methods. The difference affects local-time operations:

- `Date.prototype.toString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.getHours()` and other local getters
- Local `Date` constructors and setters
- Operations that implicitly convert dates to strings, including `Array.prototype.sort()` called without a comparator

Consider the following operations on a `mongod` that runs with `TZ=America/New_York`:

```
db.events.insertOne( {
  name: "before-opening",
  occurredAt: ISODate("2024-01-15T13:30:00Z")
} )

db.events.find( {
  $expr: {
    $function: {
      body: function(date) { return date.getHours() < 9; },
      args: [ "$occurredAt" ],
      lang: "js"
    }
  }
} )
```

In earlier versions, `13:30` UTC is `08:30` in New York, so `getHours()` returns `8` and the query matches the document. Starting in MongoDB 9.0, `getHours()` returns `13` and the query does not match. The operation succeeds in both versions because the stored date remains valid, so the difference is not reported as an error. Deployments that run mixed binary versions can return different results for the same JavaScript.

Upgrade Impact

Before you upgrade to MongoDB 9.0, review the JavaScript in your `$function`, `$accumulator`, `$where`, and `mapReduce` code for local `Date` usage, and do not rely on the `mongod` host time zone.

To evaluate dates in a named time zone, use an aggregation date operator with an explicit `timezone` argument instead of JavaScript. The following query returns the same results in all versions:

```
db.events.find( {
  $expr: {
    $lt: [ { $hour: { date: "$occurredAt",
                      timezone: "America/New_York" } }, 9 ]
  }
}
```

For JavaScript that requires deterministic comparison or serialization, use `getTime()`, `toISOString()`, or the `getUTC*()` methods rather than local `Date` methods or implicit string conversion. Pass an explicit comparator to `Array.prototype.sort()` when the sorted values can contain dates.

To learn more, see [Server-side JavaScript](#).

Server-Side JavaScript Unavailable on ppc64le

Starting in MongoDB 9.0, server-side JavaScript is not available on the `ppc64le` architecture. The `mongod` and `mongos` binaries for `ppc64le` do not include a JavaScript engine. As a result, `$function`, `$accumulator`, `$where`, and `mapReduce` operations fail on that architecture. Earlier versions ran these operations on `ppc64le`.

You cannot enable server-side JavaScript on `ppc64le` with a configuration file setting or a command-line option.

Upgrade Impact

Before you upgrade a `ppc64le` deployment to MongoDB 9.0, identify applications that use `$function`, `$accumulator`, `$where`, or `mapReduce`. Rewrite these operations to use [aggregation pipeline](#) stages and operators that do not require server-side JavaScript. You can also run the operations on a deployment that uses a different architecture.

To learn more, see [Server-side JavaScript](#).

Security

Queryable Encryption Prefix, Suffix, and Substring Public Preview to GA Migration

MongoDB 9.0 marks the GA of prefix, suffix, and substring queries on encrypted string fields in Queryable Encryption enabled collections. The GA feature is incompatible with the Public Preview version released in MongoDB 8.2, which should not be used now that the feature is GA.

To use prefix, suffix, or substring queries with Queryable Encryption, MongoDB must be version 9.0 or later, with 9.0 compatible drivers. If you are still using the Public Preview, MongoDB must remain on version 8.2 or 8.3, with 8.2 or 8.3 compatible drivers.

MongoDB 9.0 drivers can decrypt data created with MongoDB 8.2 or 8.3 drivers. For upgrade options, see the following sections.

Start Fresh (Preferred)

If possible, create new collections rather than migrating collections created with Public Preview functionality:

1. Upgrade MongoDB server and drivers to 9.0.

2. Configure a new encrypted collection with a different name from the previous collection.
3. Insert new data, or an unencrypted version of the existing data if you have a local copy.
4. Drop the previous collection.

Migrate Existing Data

If you cannot use new data or do not have an unencrypted version of the existing data:

1. Upgrade MongoDB server and drivers to 9.0
2. Using a 9.0 compatible driver, query the encrypted collection to decrypt it.
3. Save the output locally.
4. Configure a new encrypted collection and ingest the data.

Warning:

- The `mongoexport` and `mongodump` operations do not decrypt the collection. You must query the collection from a driver to output decrypted data.
- MongoDB 9.0 compatible drivers can't query encrypted fields on data encrypted for MongoDB 8.2 Public Preview query types. To decrypt data, either query an unencrypted field or query the entire collection.

Backward-Incompatible Features

The following sections provide information for removing backward-incompatible features from your deployment. If you are downgrading from MongoDB 9.0 to an earlier version, review the following sections to ensure that your deployment runs successfully after downgrading.

Expressions in Views

In MongoDB 9.0, `$convert` can convert an object to `binData`. For details, see [Convert an Object to binData](#).

If you create a view that uses this conversion, queries on that view return an error after you downgrade to an earlier version.

Before you downgrade from 9.0, update or drop any views that use this conversion.